

Step 1: Create Custom Object

Go to:

Setup → Object Manager → Create → Custom Object

Create:

Property	Value
Label	Emp
Object Name	Emp

Step 2: Create Fields

Inside `Employee__c` create:

Field Label	Data Type	API Name
Emp Name	Text	Emp_Name__c
Emp ID	Number	Emp_ID__c
Salary	Currency	Salary__c
Email	Email	Email__c
Department	Picklist	Department__c
Joining Date	Date	Joining_Date__c

Step 3: Create Apex Class

Go to:

Setup → Apex Classes → New

Paste:

```
public with sharing class EmployeeController {
```

```

@AuraEnabled
public static String saveEmployee(
    String empName,
    Integer empld,
    Decimal salary,
    String email,
    String department,
    Date joiningDate
){

    // Name Validation
    if(String.isBlank(empName) || empName.length() < 3) {

        return 'Employee Name must contain at least 3 characters';
    }

    // Employee ID Validation
    if(empld <= 0) {

        return 'Employee ID must be greater than 0';
    }

    // Unique Employee ID
    List<Emp__c> empList = [

        SELECT Id
        FROM Emp__c
        WHERE Emp_ID__c = :empld
    ];

    if(empList.size() > 0) {

        return 'Employee ID already exists';
    }

    // Salary Validation
    if(salary < 10000 || salary > 500000) {

        return 'Salary must be between 10,000 and 500,000';
    }

    // Email Validation
    Pattern pattern = Pattern.compile(
        '^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$'
    );

    Matcher matcher = pattern.matcher(email);

```

```

    if(!matcher.matches()) {

        return 'Invalid Email Format';
    }

    // Department Validation
    if(String.isBlank(department)) {

        return 'Please select department';
    }

    // Joining Date Validation
    if(joiningDate > Date.today()) {

        return 'Joining Date cannot be future date';
    }

    // Save Record
    Emp__c emp = new Emp__c();

    emp.Emp_Name__c = empName;
    emp.Emp_ID__c = empId;
    emp.Salary__c = salary;
    emp.Email__c = email;
    emp.Department__c = department;
    emp.Joining_Date__c = joiningDate;

    insert emp;

    return 'Employee Saved Successfully';
}
}

```

Step 4: Create LWC

Fix Step-by-Step

Step 1: Install Salesforce CLI

Official website open karo:

https://developer.salesforce.com/tools/salesforcecli?utm_source=chatgpt.com

[Salesforce CLI Download](#)

Download:

- Windows x64 installer

Install normally.

Step 2: Restart VS Code

Completely close:

- VS Code
- Terminal

Then reopen VS Code.

Step 3: Check CLI Installed or Not

VS Code terminal me run karo:

```
sf --version
```

Agar output aaya like:

```
@salesforce/cli
```

to install successful hai.

Step 1: Installation pr error aya toh

Installer complete hone do.

Step 2: Find Salesforce CLI Folder

Open:

C:\Program Files\sfb\bin

Check karo kya:

- `sf.cmd`
- `sf.exe`

present hai.

Step 3: Copy Path

Copy this:

C:\Program Files\sfb\bin

Step 4: Open Environment Variables

Windows Search me search:

environment variables

Open:

Edit the system environment variables

Then:

Environment Variables

Step 5: Edit PATH

Under:

User variables

select:

Path

Click:

Edit

Then:

New

Paste:

C:\Program Files\sfb\bin

Click:

- OK
 - OK
 - OK
-

Step 6: Restart VS Code

Close:

- VS Code
- Terminal

Open again.

Step 7: Verify

Run:

`sf --version`

If installation successful, output aayega.

Step 1: Installation pr error aya toh(Ubuntu)

Terminal me run karo:

```
sf --version
```

Agar:

command not found

aata hai, to PATH add karna padega.

Step 2: Find sf Binary Location

Run:

```
find / -name sf 2>/dev/null
```

Usually path hota hai:

```
/usr/local/bin/sf
```

ya:

```
/snap/bin/sf
```

Step 3: Open bashrc

Run:

```
nano ~/.bashrc
```

Step 4: Add PATH

File ke end me add karo:

```
export PATH=$PATH:/usr/local/bin
```

Agar path `/snap/bin` hai to:

```
export PATH=$PATH:/snap/bin
```

Step 5: Save File

Press:

Ctrl + O

Enter

Ctrl + X

Step 6: Reload bashrc

Run:

```
source ~/.bashrc
```

Step 7: Verify

Run again:

```
sf --version
```

Agar version aa gaya → done 

Go to:

VS Code

Step 1: Open VS Code

Install:

- VS Code
 - Salesforce CLI
 - Salesforce Extension Pack
-

Step 2: Create Salesforce Project

Press:

Ctrl + Shift + P

Search:

SFDX: Create Project

Choose:

Standard

Project name:

EmployeeProject

Step 3: Authorize Org

Press:

Ctrl + Shift + P

Search:

SFDX: Authorize an Org

Login to Salesforce.

Step 4: Create LWC

Press:

Ctrl + Shift + P

Search:

SFDX: Create Lightning Web Component

Component Name:

employeeManagement

Folder:

force-app/main/default/lwc

Now 3 files automatically create hong:

- employeeManagement.html
- employeeManagement.js
- employeeManagement.js-meta.xml

employeeManagement.html

<template>

<lightning-card title="Employee Management">

<div class="slds-p-around_medium">

<lightning-input
label="Employee Name"
onchange={handleName}>
</lightning-input>

<lightning-input
type="number"
label="Employee ID"
onchange={handleId}>
</lightning-input>

<lightning-input
type="number"

```

        label="Salary"
        onchange={handleSalary}>
    </lightning-input>

    <lightning-input
        type="email"
        label="Email"
        onchange={handleEmail}>
    </lightning-input>

    <lightning-combobox
        label="Department"
        options={deptOptions}
        onchange={handleDepartment}>
    </lightning-combobox>

    <lightning-input
        type="date"
        label="Joining Date"
        onchange={handleDate}>
    </lightning-input>

    <br/>

    <lightning-button
        label="Save Employee"
        variant="brand"
        onclick={saveEmployee}>
    </lightning-button>

</div>

</lightning-card>

</template>

employeeManagement.js -
import { LightningElement } from 'lwc';

import saveEmployeeData
from '@salesforce/apex/EmployeeController.saveEmployee';

import { ShowToastEvent }
from 'lightning/platformShowToastEvent';

export default class EmployeeManagement extends LightningElement {

```

```
empName;
empld;
salary;
email;
department;
joiningDate;

deptOptions = [

    { label: 'IT', value: 'IT' },
    { label: 'HR', value: 'HR' },
    { label: 'Sales', value: 'Sales' }

];

handleName(event) {

    this.empName = event.target.value;
}

handleId(event) {

    this.empld = event.target.value;
}

handleSalary(event) {

    this.salary = event.target.value;
}

handleEmail(event) {

    this.email = event.target.value;
}

handleDepartment(event) {

    this.department = event.target.value;
}

handleDate(event) {

    this.joiningDate = event.target.value;
}

saveEmployee() {

    saveEmployeeData({
```

```

        empName: this.empName,
        empld: this.empld,
        salary: this.salary,
        email: this.email,
        department: this.department,
        joiningDate: this.joiningDate
    })

    .then(result => {

        this.dispatchEvent(

            new ShowToastEvent({

                title: 'Success',
                message: result,
                variant: 'success'

            })
        );
    })

    .catch(error => {

        this.dispatchEvent(

            new ShowToastEvent({

                title: 'Error',
                message: error.body.message,
                variant: 'error'

            })
        );
    });
}
}

```

employeeManagement.js-meta.xml -

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<LightningComponentBundle
  xmlns="http://soap.sforce.com/2006/04/metadata">

  <apiVersion>60.0</apiVersion>

  <isExposed>true</isExposed>

  <targets>

    <target>lightning__AppPage</target>
    <target>lightning__HomePage</target>
    <target>lightning__RecordPage</target>

  </targets>

</LightningComponentBundle>
```

Step 1: Save All Files

Press:

Ctrl + Shift + S

ya individually save karo.

Step 2: Deploy LWC to Salesforce

VS Code left side Explorer me:

```
force-app
├── main
│   ├── default
│   │   └── lwc
│   │       └── employeeManagement
```

folder pe right click karo:

SFDX: Deploy Source to Org

Wait for success message.

Step 3: Open Salesforce

Go to:

Setup → Lightning App Builder

Step 4: Create New App Page

Click:

New → App Page

Page name:

Employee Management

Select any layout → Finish.

Step 5: Add Your LWC

Left side Custom Components me:

employeeManagement

dikhega.

Usko drag & drop karo page pe.

Step 6: Save and Activate

Click:

- Save
- Activate

Then:

- Assign as Org Default or App Default

Step 7: Open Page

Open Page

After activation:

Click top left:

← Back

Then:

App Launcher (9 dots)

Search:

Employee Management

Open it.

DISPLAY KRNE KE LIYE

Step 1: Update Apex Class

Ye method existing Apex class me add karo:

```
@AuraEnabled(cacheable=true)
```

```
public static List<Emp__c> getEmployees() {
```

```
    return [
```

```
SELECT Id,  
       Emp_Name__c,  
       Emp_ID__c,  
       Salary__c,  
       Email__c,  
       Department__c,  
       Joining_Date__c  
FROM Emp__c  
];  
}
```

Step 2: Update employeeManagement.js

Purana JS replace karo:

```
import { LightningElement, wire, track } from 'lwc';
```

```
import saveEmployeeData
```

```
from '@salesforce/apex/EmployeeController.saveEmployee';
```

```
import getEmployees
```

```
from '@salesforce/apex/EmployeeController.getEmployees';
```

```
import { ShowToastEvent }
```

```
from 'lightning/platformShowToastEvent';
```

```
import { refreshApex }
```

```
from '@salesforce/apex';
```

```
export default class EmployeeManagement extends LightningElement {
```

```
    empName;
```

```
    empld;
```

```
    salary;
```

```
    email;
```

```
    department;
```

```
    joiningDate;
```

```
    @track employeeList;
```

```
    wiredResult;
```

```
    columns = [
```

```
        { label: 'Name', fieldName: 'Emp_Name__c' },
```

```
        { label: 'Employee ID', fieldName: 'Emp_ID__c' },
```

```
        { label: 'Salary', fieldName: 'Salary__c' },
```

```
        { label: 'Email', fieldName: 'Email__c' },
```

```
        { label: 'Department', fieldName: 'Department__c' },
```

```
        { label: 'Joining Date', fieldName: 'Joining_Date__c' }
```

```
    ];
```

```
    deptOptions = [
```

```
{ label: 'IT', value: 'IT' },  
{ label: 'HR', value: 'HR' },  
{ label: 'Sales', value: 'Sales' }
```

```
];
```

```
@wire(getEmployees)
```

```
wiredEmployees(result) {
```

```
    this.wiredResult = result;
```

```
    if(result.data) {
```

```
        this.employeeList = result.data;
```

```
    }
```

```
}
```

```
handleName(event) {
```

```
    this.empName = event.target.value;
```

```
}
```

```
handleId(event) {
```

```
    this.empld = event.target.value;
```

```
}
```

```
handleSalary(event) {
```

```
    this.salary = event.target.value;
```

```
}
```

```
handleEmail(event) {
```

```
    this.email = event.target.value;
```

```
}
```

```
handleDepartment(event) {
```

```
    this.department = event.target.value;
```

```
}
```

```
handleDate(event) {
```

```
    this.joiningDate = event.target.value;
```

```
}
```

```
saveEmployee() {
```

```
    saveEmployeeData({
```

```
        empName: this.empName,
        empld: this.empld,
        salary: this.salary,
        email: this.email,
        department: this.department,
        joiningDate: this.joiningDate
    })

    .then(result => {

        this.dispatchEvent(

            new ShowToastEvent({

                title: 'Success',
                message: result,
                variant: 'success'

            })
        );

        refreshApex(this.wiredResult);
    })

    .catch(error => {
```

```
this.dispatchEvent(  
  
  new ShowToastEvent({  
  
    title: 'Error',  
    message: error.body.message,  
    variant: 'error'  
  
  })  
);  
});  
}  
}
```

Step 3: Update employeeManagement.html

Niche datatable add karo:

```
<template>
```

```
<lightning-card title="Employee Management">
```

```
<div class="slds-p-around_medium">
```

```
<lightning-input
```

```
  label="Employee Name"
```

```
        onchange={handleName}>
</lightning-input>
```

```
<lightning-input
  type="number"
  label="Employee ID"
  onchange={handleId}>
</lightning-input>
```

```
<lightning-input
  type="number"
  label="Salary"
  onchange={handleSalary}>
</lightning-input>
```

```
<lightning-input
  type="email"
  label="Email"
  onchange={handleEmail}>
</lightning-input>
```

```
<lightning-combobox
  label="Department"
  options={deptOptions}
  onchange={handleDepartment}>
</lightning-combobox>
```

```
<lightning-input  
  type="date"  
  label="Joining Date"  
  onchange={handleDate}>  
</lightning-input>
```

```
<br/>
```

```
<lightning-button  
  label="Save Employee"  
  variant="brand"  
  onclick={saveEmployee}>  
</lightning-button>
```

```
<br/><br/>
```

```
<!-- Employee Table -->
```

```
<lightning-datatable
```

```
  key-field="Id"
```

```
  data={employeeList}
```

```
  columns={columns}>
```

```
</lightning-datatable>
```

```
</div>
```

```
</lightning-card>
```

```
</template>
```

Step 4: Deploy Again

Run: in vs code terminal

sf project deploy start